Sinhala A/L ICT AI Tutor — Project Specification

Stack: React (frontend) · FastAPI (backend) · Supabase Postgres + pgvector (database) · LLM API + multilingual embeddings
Purpose of this file: the single source of truth for the project. Keep it in the repo as `docs/SPEC.md` and point your AI coding agent at it for every task.

---

1. Project Summary

A bilingual (Sinhala / English) web app for Grade 12–13 A/L ICT students. Students learn **unit by unit** or across the **whole syllabus**, in two ways: **Ask** (quick cited answers) and **Discuss** (a step-by-step teaching conversation about theory). They practise with a quiz engine that supports **MCQ, structured, and essay** questions. Every answer is grounded in verified syllabus material (RAG) and cites its source. Student and teacher feedback becomes a **reward signal** that is used to improve prompts, retrieval, and (later) models.

Core promise: answers come from approved material. When the material does not cover the question, the tutor says so instead of guessing.

1.1 Goals

- Accurate, cited answers in Sinhala and English (and mixed).
- Unit-by-unit and whole-syllabus learning, in Ask and Discuss modes.
- Exam-style practice: MCQ, structured, essay, with marking schemes and rubrics.
- Progress tracking and weak-topic detection.
- A feedback-driven improvement loop with measurable accuracy gains.

1.2 Non-goals (for the first version)

- Training or fine-tuning a large LLM from scratch.
- Native mobile apps (a responsive PWA is enough).
- Voice input and WhatsApp delivery (listed as later ideas).

---

2. Users and Roles

The system has three roles. **Students create the data, teachers make it trustworthy, and admins turn it into improvement.**

2.1 Student

Description: The learner and main user. The whole app is designed around their learning.

Can do

- Browse units and lessons and track their own progress
- Ask questions and use Discuss mode, in unit or whole-syllabus scope
- Take quizzes (MCQ, structured, essay) and exam-mode papers
- Use the mistake notebook, flashcards, study plan, and glossary
- Give feedback: thumbs up/down, reasons, "report this question," "do you agree with this mark?"

Cannot do: see other students' data, approve questions, change marks, upload content, or change settings.

Value of this role

- *To themselves:* faster understanding, exam-style practice, and a clear picture of weak topics.
- *To the system:* their feedback and quiz results are the raw data for the reward system and for finding bad answers.
- *To the project:* real usage and pilot results turn it from a demo into evidence (for example, score improvement).

2.2 Teacher

Description: The quality controller and human expert. This is the project owner, and any teacher invited later.

Can do

- Everything a student can do (to experience the app as a student)

- Upload syllabus material, notes, and past papers
- Review the question queue: approve, edit, or reject generated questions
- Review flagged answers and correct them
- Override AI-suggested marks on structured and essay answers
- Create classes, add students, and assign quizzes with deadlines
- View analytics for their own classes (weak units, struggling students, most-reported content)
- Export progress reports

Cannot do: manage other users' accounts, change prompt versions, or alter system-wide settings.

Value of this role

- *To students:* correct questions and fair marks, because a human checks the AI.
- *To the system:* teacher decisions are the highest-quality reward signal. Corrections become preference pairs for later fine-tuning, and approved questions form the question bank.
- *To other teachers:* class analytics show where to focus teaching time.
- *To the project:* teacher-reviewed content and marking-agreement scores are strong evidence of quality.

2.3 Admin

Description: The system owner and operator, usually the project owner wearing a separate hat.

Can do

- Manage users and roles (promote a student to teacher, disable accounts)
- Manage units, lessons, and the glossary structure
- Manage prompt versions and see their average reward
- View the quality dashboard: failed answers, most-reported content, cost, latency
- Configure system settings (rate limits, confidence threshold, active models)
- Review audit logs and handle data deletion requests

Cannot do (by design): routinely read private student answers. Limit admin visibility to what is needed and log access.

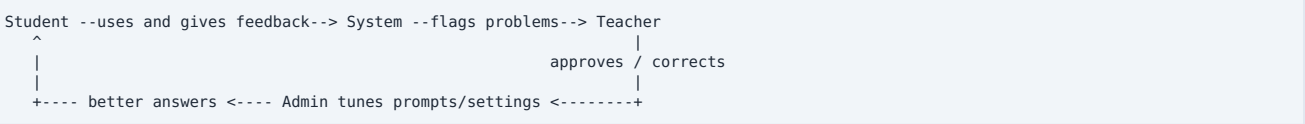
Value of this role

- *To the system:* keeps it safe, stable, and affordable, with the right people holding the right permissions.
- *To the improvement loop:* admins choose which prompts run, which settings are active, and when to roll back a bad change.
- *To the project:* demonstrates governance, security, and operations thinking, not only features.

2.4 Permission Matrix

Action	Student	Teacher	Admin
Ask / Discuss / Quiz	Yes	Yes	Yes
View own progress	Yes	Yes	Yes
Give feedback	Yes	Yes	Yes
Upload content	No	Yes	Yes
Approve / edit questions	No	Yes	Yes
Override marks	No	Yes	Yes
Create classes, assign quizzes	No	Yes	Yes
View class analytics	No	Yes (own classes)	Yes (all)
Manage users and roles	No	No	Yes
Manage prompts and settings	No	No	Yes
View quality and cost dashboard	No	Limited	Yes

2.5 How the Roles Work Together



2.6 Role Implementation Notes

- Store the role in `profiles.role` and include it in the JWT claims; check it in FastAPI with role dependencies (`require_role("teacher")`).
- Use **Row Level Security** in Supabase so a student only reads their own rows and a teacher only sees students in their own classes.
- Create the **first admin manually** in the database. Do not allow self-signup as teacher or admin; teachers are approved by an admin or join through an invite code.
- Version 1 uses one role per user. If a user later needs several roles, move to a `user_roles` table.

Example RLS policies (starting point):

```
alter table attempts enable row level security;

-- Students read and write only their own attempts
create policy "student_own_attempts" on attempts
  for all using (user_id = auth.uid());

-- Teachers read attempts of students in their classes
create policy "teacher_class_attempts" on attempts
  for select using (
    exists (
      select 1 from class_members cm
      join classes c on c.id = cm.class_id
      where cm.student_id = attempts.user_id
        and c.teacher_id = auth.uid()
    )
  );

-- Only teachers and admins can change question status
create policy "teacher_manage_questions" on questions
  for update using (
    exists (select 1 from profiles p
            where p.id = auth.uid() and p.role in ('teacher','admin'))
  );
```

Apply the same pattern to `mistakes`, `interactions`, `feedback`, `mastery`, and `flashcards`.

---

3. Functional Requirements

3.1 Authentication and profiles

- FR-A1: Sign up / log in (email + password; optional Google login).
- FR-A2: Roles: student, teacher, admin.
- FR-A3: Profile with name, grade, preferred language (Sinhala / English / mixed).

3.2 Unit and lesson navigation

- FR-U1: Unit dashboard with Sinhala and English unit names, lesson lists, progress bars, average scores.
- FR-U2: Each unit offers **Ask**, **Discuss**, and **Quiz** entry points.
- FR-U3: Lesson summary (one-tap revision notes generated from source material).

3.3 Ask mode (quick answers)

- FR-Q1: Free-text question in Sinhala, English, or mixed.
- FR-Q2: **Scope toggle:** *This unit* (filtered retrieval) or *Whole syllabus* (unfiltered retrieval).
- FR-Q3: Answer returned in the student's language with citations (unit, lesson, passage).
- FR-Q4: Buttons: *Explain simpler*, *Give an example*, *Answer in Sinhala*, *Answer in English*.
- FR-Q5: If retrieval confidence is low, show a "not found in the syllabus material" message and a confidence warning.
- FR-Q6: Scope suggestions: "This looks like Unit 4 — search all units?" / "Open Unit 4 for more detail?"
- FR-Q7: Source panel showing the exact passages used.

3.4 Discuss mode (theory teaching)

- FR-D1: Progressive explanation of a lesson or concept, with examples.
- FR-D2: The tutor asks short check questions, gives hints before answers, adapts to replies.
- FR-D3: Student controls: *explain again simpler*, *exam-style example*, *compare with X*.
- FR-D4: Ends with a summary and offers a quick quiz on the discussed concepts.
- FR-D5: Works in both scopes (unit and whole syllabus).
- FR-D6: Concepts covered and struggled with are stored for the weak-topic tracker.

3.5 Quiz engine

- FR-Z1: Quiz setup: scope (unit / multi-unit / whole syllabus), lessons, question types and counts, difficulty, language.

- FR-Z2: **MCQ** — 5 options, one correct answer, explanation for each option, automatic marking.
- FR-Z3: **Structured** — scenario with parts (a), (b), (c), marks per part, stored marking scheme, AI-assisted marking.
- FR-Z4: **Essay** — total marks, rubric with weighted criteria, AI-assisted marking with per-criterion feedback.
- FR-Z5: **Exam mode** — timer and paper-style layout (MCQ section; structured + essay section). *Verify the layout and mark allocations against the current official paper format.*
- FR-Z6: Model answers and explanations after submission.
- FR-Z7: **Approved question bank** — generated questions are reviewed by a teacher before entering the bank; quizzes draw from approved questions first.
- FR-Z8: No repeats — track which questions a student has already seen.
- FR-Z9: Balanced sampling across lessons (and across units for whole-syllabus quizzes).
- FR-Z10: "Report this question" button.

3.6 Past papers

- FR-P1: Browse past paper questions by year and unit; attempt and get marked.
- FR-P2: Frequency analysis: which units and lessons appear most in past papers.

3.7 Progress and revision

- FR-R1: Weak-topic tracker: accuracy by unit, lesson, and question type.
- FR-R2: **Mistake notebook:** every wrong answer saved automatically.
- FR-R3: "Revise my mistakes" quiz built from the notebook.
- FR-R4: Spaced-repetition flashcards for key definitions.
- FR-R5: Study planner: exam date in, weekly plan out, based on weak units.
- FR-R6: Streaks and badges (class leaderboard optional, teacher can disable).

3.8 Extra learning tools

- FR-X1: **Sinhala–English glossary** with search and short definitions.
- FR-X2: **Diagram support:** logic gates, ER diagrams, network topologies, flowcharts (Mermaid or SVG).
- FR-X3: **Code/query tracer** for Python, SQL, HTML (step-by-step explanation; sandboxed execution is an optional later upgrade).

3.9 Feedback and reward

- FR-F1: thumbs up / thumbs down on every answer; on thumbs down, reason chips: *Wrong answer · Not clear · Wrong language · Too long · Missing detail · Not from my syllabus*; optional comment.
- FR-F2: After quiz questions: "Was this explanation helpful?"; on marking: "Do you agree with this mark?"
- FR-F3: Teacher review actions: approve, edit, reject answers and questions; corrected version stored beside the original.
- FR-F4: Implicit signals logged: repeated questions, "explain simpler" clicks, source-panel opens, later quiz improvement.
- FR-F5: Reward score computed per interaction (see section 11).
- FR-F6: Admin view of prompt versions and their average reward.

3.10 Teacher and admin tools

- FR-T1: Upload PDFs/notes/past papers; ingestion runs in the background with status.
- FR-T2: Review queue for generated questions and flagged answers.
- FR-T3: Mark override for structured/essay answers.
- FR-T4: Classes and assignments: create a class, assign a quiz with a deadline, view per-student results.
- FR-T5: Class analytics: weak units and lessons, most-reported content.
- FR-T6: Progress report export (PDF).
- FR-T7: Quality dashboard: failed answers, most-reported questions, unanswered topics.

---

4. Non-Functional Requirements

- **Language quality:** natural, student-level Sinhala; technical terms kept in English where textbooks do.
- **Performance:** first token of an answer within ~3 seconds (stream responses); quiz generation can be asynchronous.
- **Mobile-first:** works well on small screens and low bandwidth (PWA, lazy loading, light pages).
- **Reliability:** graceful fallback when the LLM or database is slow.
- **Security and privacy:** see section 13.
- **Cost control:** caching, approved question bank, per-user rate limits.
- **Observability:** log every interaction with prompt version, retrieved chunks, latency, and reward.

---

5. Core Theory — How It Works

5.1 Why not just ask an LLM?

A general LLM answers from patterns learned in training. It can sound confident while being wrong (**hallucination**), it may not match the Sri Lankan NIE syllabus, and its Sinhala can be uneven. For a tutor, wrong answers are harmful. The fix is to make the model answer from **your approved material**.

5.2 Retrieval-Augmented Generation (RAG)

RAG splits the task into **retrieve** and **generate**:

1. **Chunking.** Split syllabus documents into small passages (about 300–500 tokens), keeping each heading with its text.
2. **Embedding.** An embedding model converts each chunk into a vector (a list of numbers, e.g. 1024 values) so that chunks with similar *meaning* end up close together in vector space. A **multilingual** model (such as BGE-M3 or multilingual-e5) places a Sinhala sentence and its English equivalent near each other.
3. **Vector search.** The student's question is embedded the same way. The database finds the chunks whose vectors are closest, using **cosine similarity**. Metadata filters (unit, lesson) narrow the search for unit mode.
4. **Reranking (optional but valuable).** A reranker re-scores the top 20 chunks against the question and keeps the best 5–8.
5. **Grounded generation.** The LLM receives the question plus the retrieved chunks and an instruction to answer **only** from them, cite them, and say "not found" otherwise.

Why chunk metadata matters: each chunk stores `unit`, `lesson`, `language`, `source_type`, `year`. This is what makes unit mode a simple filter on the same pipeline as whole-syllabus mode.

5.3 Ask mode vs Discuss mode

Both use retrieval. They differ in the **system prompt and conversation policy**:

- *Ask:* concise, cited, direct.
- *Discuss:* teach in small steps, ask check questions, give hints before answers, adapt to what the student says. Same grounding rule.

5.4 Structured outputs for quizzes

Quiz questions are generated as **JSON that must match a schema**. The backend validates every response with Pydantic models. Invalid output is retried or discarded. This turns free text into reliable data that can be stored, marked, and analysed.

5.5 AI-assisted marking

- **MCQ:** deterministic comparison with the stored answer.
- **Structured:** the model compares the student's answer with the stored **marking points** (keywords and marks per point) and returns marks per point with reasons.
- **Essay:** the model scores each **rubric criterion** and explains it. AI marks are treated as **suggestions** until your evaluation shows good agreement with teacher marks. Teachers can always override.

5.6 Learning analytics and mastery

Track a mastery score per student per lesson. A simple, explainable method is an **exponential moving average** of recent correctness: `mastery_new = 0.7 × mastery_old + 0.3 × latest_score` Lessons with low mastery drive revision suggestions and the study planner.

5.7 Spaced repetition

For flashcards use **SM-2**: after each review, the student rates recall; the interval until the next review grows when recall is good and resets when it is poor. This schedules key definitions at the moment they are about to be forgotten.

5.8 Feedback as a reward signal

A **reward** is a number that says how good an answer was. Combine teacher judgement, student ratings, and learning outcomes (section 11). Use the reward to:

1. compare **prompt versions**,
2. run **A/B tests** with a **multi-armed bandit** (e.g. Thompson sampling) so better settings get more traffic,
3. **score chunks** so helpful passages rank higher and harmful ones are flagged,
4. choose between candidate answers (**best-of-N** with a small reward model),
5. later, build **preference pairs** (chosen vs rejected answer) for **DPO** fine-tuning of a small open model.

This is reinforcement-learning-style improvement of the **system**, without retraining a large LLM.

5.9 Evaluation metrics

- **Retrieval:** Recall@k (right chunk in top k), MRR.
- **Answer quality:** correct / partial / wrong, graded by a teacher on a fixed test set.
- **Faithfulness:** does the answer stay inside the retrieved context? Hallucination rate = answers given where the material has no answer.
- **Quiz quality:** % of generated questions approved without edits.
- **Marking agreement:** AI marks vs teacher marks (e.g. mean absolute difference, agreement rate).
- **Learning impact:** quiz score change over time in a pilot group.

---

6. Technology Stack

Layer	Choice	Reason
Frontend	React + Vite + TypeScript	Fast dev, strong ecosystem, good for AI-assisted coding
UI	Tailwind CSS + shadcn/ui	Quick, consistent components
Routing / data	React Router, TanStack Query	Clean routing and server-state caching
Client state	Zustand	Small global state (scope, mode, language)
i18n	i18next	Sinhala / English UI strings
Font	Noto Sans Sinhala	Correct Sinhala rendering
Charts	Recharts	Progress dashboards
Diagrams	Mermaid	Flowcharts, ER diagrams
Backend	FastAPI (Python)	Async, typed (Pydantic), best fit for AI/ML libraries
Validation	Pydantic v2	Request/response and LLM JSON schemas
DB access	SQLAlchemy 2 (async) + asyncpg, or supabase-py	Typed queries; use SQL for vector search
Database	**Supabase (Postgres + pgvector)**	See section 7
Auth	Supabase Auth (JWT verified in FastAPI)	No custom auth to build
File storage	Supabase Storage	Uploaded PDFs
Embeddings	BGE-M3 or multilingual-e5	Multilingual, handles Sinhala/English
Reranker	bge-reranker (optional)	Better top results
LLM	An LLM API with good Sinhala output	Called only from the backend
OCR	Tesseract with Sinhala data (+ manual cleaning)	Scanned PDFs
Background jobs	FastAPI BackgroundTasks first; ARQ/Celery + Redis later	Ingestion, quiz generation
Testing	pytest, Vitest, Playwright	Backend, frontend, end-to-end
Deployment	Frontend: Vercel/Netlify · Backend: Render/Railway/Fly.io or Docker VPS	Simple hosting

---

7. Database Decision: Supabase vs MongoDB vs SQL

7.1 Comparison

Need	Supabase (Postgres + pgvector)	MongoDB (Atlas)	Plain SQL DB (MySQL/Postgres)
Users, classes, assignments, attempts (relational data)	Excellent (joins, constraints)	Possible, but relations are manual	Excellent
Analytics: accuracy per unit/lesson/student, class reports	Excellent (SQL aggregates)	Harder (aggregation pipelines)	Excellent
Vector search for RAG	Built in (pgvector)	Atlas Vector Search (Atlas only)	Needs extra extension or separate vector DB
Flexible question JSON (MCQ/structured/essay)	JSONB columns	Natural fit	JSON columns (varies)
Auth, file storage, row-level security	Included	Build or add services	Build or add services
Number of services to run	One	One + separate auth/storage	Several
FastAPI integration	Good (SQLAlchemy/asyncpg or supabase-py)	Good (Motor/Beanie)	Good

7.2 Recommendation: Supabase (managed Postgres with pgvector)

Reasons:

1. **One database for everything:** relational data, JSON question content (JSONB), and vector embeddings. Unit-filtered vector search is a single SQL query with a `WHERE unit_id = ...`.
2. **Analytics are central to this app** (weak topics, class reports, progress). SQL is much better at this than a document database.
3. **Auth, storage, and row-level security come included,** which saves weeks of work, especially important when vibe coding.
4. **Easy to move later:** it is standard Postgres, so you are not locked in.

When MongoDB would be better: if your data were mostly unstructured documents with no analytics. That is not this project.

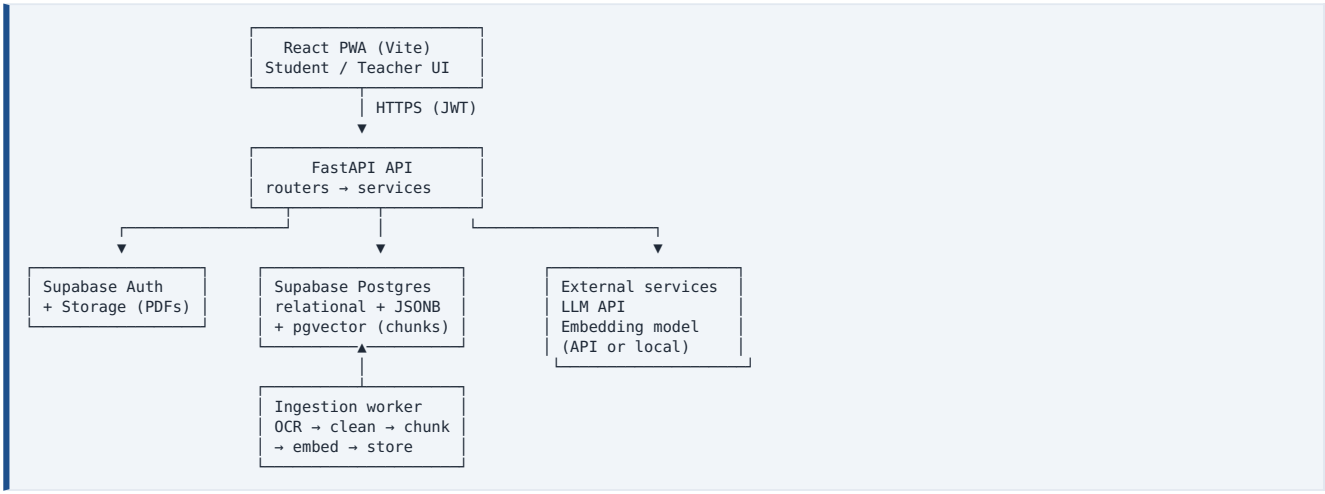
Practical notes:

- Check the current free-tier limits (database size, storage, pausing of inactive projects) before you rely on it for the pilot.
- Run vector search through SQL from FastAPI. Use an **HNSW index** on the embedding column with cosine distance.
- Keep embedding dimension consistent with your model (BGE-M3 produces 1024 dimensions).

---

8. System Architecture

8.1 High-level diagram



8.2 Request flow: Ask mode

1. Student sends `{question, scope, unit_id?, lesson_id?, mode: "ask", language}`.
2. API verifies the JWT and loads the user.

3. Detect language; embed the question.
4. Vector search (with unit filter if scope = unit), then rerank.
5. Confidence check on the best similarity score. If too low → "not found in material."
6. Build the prompt: system rules + retrieved chunks + question.
7. Call the LLM, **stream** tokens to the client.
8. Save the interaction (question, chunk IDs, prompt version, answer, latency).
9. Client shows the answer, sources, and feedback buttons.

8.3 Request flow: Discuss mode

Same retrieval, but the prompt includes the **conversation history** and a teaching policy ("explain one idea at a time, ask a check question, give hints first"). A small `lesson_progress` record stores concepts covered and concepts the student struggled with.

8.4 Request flow: Quiz generation

1. Student/teacher selects scope, types, counts, difficulty, language.
2. Backend first draws **approved** questions from the bank (not seen before, balanced across lessons).
3. If more are needed, sample chunks across lessons and ask the LLM for JSON questions.
4. Validate against Pydantic schemas; auto-checks (one correct MCQ answer, marks add up, marking scheme present).
5. Save as `draft` and send to the teacher review queue; students only receive approved questions (or drafts marked "unreviewed", if you choose to allow that).

8.5 Flow: Marking

- MCQ: compare option.
- Structured/essay: send the student's answer, the question, and the marking scheme/rubric to the LLM; get JSON with marks and feedback; store as **suggested marks**; teacher may override.

8.6 Flow: Ingestion

Teacher uploads a PDF → stored in Supabase Storage → job created → OCR/text extraction → cleaning (fix Sinhala characters) → chunk by headings → attach metadata → embed → insert into `chunks` → status `ready`.

8.7 Flow: Feedback loop

Interaction logged → feedback and implicit signals attached → reward computed (nightly job or on event) → dashboards update → weekly review: lowest-reward answers, most-reported questions, bad chunks → fix content or prompts → compare reward and test-set accuracy.

---

9. Database Schema (Postgres)

```
create extension if not exists vector;

-- Profiles (linked to Supabase auth.users)
create table profiles (
  id uuid primary key references auth.users(id) on delete cascade,
  full_name text,
  role text check (role in ('student','teacher','admin')) default 'student',
  grade int,
  preferred_language text default 'si',   -- si | en | mixed
  created_at timestamptz default now()
);

create table units (
  id serial primary key,
  name_si text not null,
  name_en text not null,
  position int not null
);

create table lessons (
  id serial primary key,
  unit_id int references units(id) on delete cascade,
  name_si text not null,
  name_en text not null,
  position int not null
);

create table documents (
  id uuid primary key default gen_random_uuid(),
  title text,
  source_type text,                -- syllabus | notes | past_paper | teacher_guide
  year int,
  storage_path text,
  status text default 'pending',    -- pending | processing | ready | failed
```

```
  uploaded_by uuid references profiles(id),
  created_at timestamptz default now()
);

create table chunks (
  id uuid primary key default gen_random_uuid(),
  document_id uuid references documents(id) on delete cascade,
  unit_id int references units(id),
  lesson_id int references lessons(id),
  language text,                    -- si | en | mixed
  content text not null,
  embedding vector(1024),
  quality_score float default 0,    -- from feedback credit
  created_at timestamptz default now()
);
create index on chunks using hnsw (embedding vector_cosine_ops);
create index on chunks (unit_id, lesson_id);

create table glossary (
  id serial primary key,
  term_si text,
  term_en text,
  definition_si text,
  definition_en text,
  unit_id int references units(id)
);

create table questions (
  id uuid primary key default gen_random_uuid(),
  type text check (type in ('mcq','structured','essay')),
  unit_id int references units(id),
  lesson_id int references lessons(id),
  difficulty text,                  -- easy | medium | hard
  language text,
  content jsonb not null,           -- question text, options/parts/rubric, marking scheme, model answer
  source_chunk_ids uuid[],
  status text default 'draft',      -- draft | approved | rejected
  reviewed_by uuid references profiles(id),
  report_count int default 0,
  created_at timestamptz default now()
);

create table quizzes (
  id uuid primary key default gen_random_uuid(),
  user_id uuid references profiles(id),
  scope text,                       -- unit | multi | all
  unit_ids int[],
  settings jsonb,
  exam_mode boolean default false,
  created_at timestamptz default now()
);

create table quiz_questions (
  quiz_id uuid references quizzes(id) on delete cascade,
  question_id uuid references questions(id),
  position int,
  primary key (quiz_id, question_id)
);

create table attempts (
  id uuid primary key default gen_random_uuid(),
  quiz_id uuid references quizzes(id),
  question_id uuid references questions(id),
  user_id uuid references profiles(id),
  answer jsonb,
  ai_marks jsonb,                   -- per-part / per-criterion suggestions
  teacher_marks jsonb,              -- overrides
  score float,
  max_score float,
  time_taken_sec int,
  created_at timestamptz default now()
);

create table mistakes (
  id uuid primary key default gen_random_uuid(),
  user_id uuid references profiles(id),
  question_id uuid references questions(id),
  revised boolean default false,
  created_at timestamptz default now()
);

create table mastery (
  user_id uuid references profiles(id),
  lesson_id int references lessons(id),
  score float default 0,
  updated_at timestamptz default now(),
  primary key (user_id, lesson_id)
);

create table flashcards (
  id uuid primary key default gen_random_uuid(),
  user_id uuid references profiles(id),
  glossary_id int references glossary(id),
  ease float default 2.5,
  interval_days int default 0,
  due_at timestamptz default now()
);
```

```
create table prompt_versions (
  id serial primary key,
  name text,                        -- ask_v1, discuss_v3, ...
  mode text,                        -- ask | discuss | quiz_gen | marking
  body text,
  uses int default 0,
  avg_reward float default 0,
  active boolean default true
);

create table interactions (
  id uuid primary key default gen_random_uuid(),
  user_id uuid references profiles(id),
  mode text,                        -- ask | discuss
  scope text,                       -- unit | all
  unit_id int,
  question text,
  chunk_ids uuid[],
  prompt_version_id int references prompt_versions(id),
  answer text,
  confidence float,
  latency_ms int,
  reward float,
  review_status text default 'none',-- none | approved | edited | rejected
  corrected_answer text,
  created_at timestamptz default now()
);

create table feedback (
  id uuid primary key default gen_random_uuid(),
  interaction_id uuid references interactions(id) on delete cascade,
  user_id uuid references profiles(id),
  rating int check (rating in (-1, 1)),
  reasons text[],
  comment text,
  created_at timestamptz default now()
);

create table classes (
  id uuid primary key default gen_random_uuid(),
  teacher_id uuid references profiles(id),
  name text
);
create table class_members (
  class_id uuid references classes(id) on delete cascade,
  student_id uuid references profiles(id) on delete cascade,
  primary key (class_id, student_id)
);
create table assignments (
  id uuid primary key default gen_random_uuid(),
  class_id uuid references classes(id) on delete cascade,
  quiz_settings jsonb,
  due_at timestamptz
);

create table preference_pairs (          -- for DPO later
  id uuid primary key default gen_random_uuid(),
  question text,
  chosen text,
  rejected text,
  created_at timestamptz default now()
);
```

Enable **Row Level Security** on user-owned tables (`attempts`, `mistakes`, `interactions`, `feedback`, `mastery`, `flashcards`) so students only see their own rows; teachers see rows for their classes.

Unit-filtered vector search (example):

```
select id, content, unit_id, lesson_id,
       1 - (embedding <=> :query_embedding) as similarity
from chunks
where (:unit_id is null or unit_id = :unit_id)
order by embedding <=> :query_embedding
limit 20;
```

---

10. Backend (FastAPI)

10.1 Folder structure

```
backend/
├─ app/
│  ├─ main.py                # app creation, CORS, routers
│  ├─ core/
│  │  ├─ config.py           # env settings (Pydantic Settings)
│  │  ├─ security.py         # JWT verification, role checks
│  │  └─ logging.py
│  ├─ db/
│  │  ├─ session.py          # async engine/session
│  │  └─ models.py           # SQLAlchemy models
│  ├─ schemas/               # Pydantic models (API + LLM JSON)
│  │  ├─ ask.py  quiz.py  question.py  feedback.py  admin.py
│  ├─ routers/
│  │  ├─ auth.py  units.py  ask.py  discuss.py  quiz.py
│  │  ├─ progress.py  glossary.py  flashcards.py
│  │  ├─ feedback.py  teacher.py  admin.py
│  ├─ services/
│  │  ├─ ingestion.py        # OCR, clean, chunk, embed, store
│  │  ├─ embeddings.py       # embedding model wrapper
│  │  ├─ retrieval.py        # vector search + rerank + confidence
│  │  ├─ llm.py              # LLM client, streaming, retries
│  │  ├─ prompts.py          # prompt loading by version
│  │  ├─ ask_service.py  discuss_service.py
│  │  ├─ quiz_generator.py   # question generation + validation
│  │  ├─ marking.py          # MCQ / structured / essay marking
│  │  ├─ mastery.py          # EMA mastery, weak topics
│  │  ├─ spaced_repetition.py # SM-2
│  │  ├─ reward.py           # reward computation, bandit
│  │  └─ analytics.py
│  ├─ workers/               # background jobs
│  └─ utils/                 # text cleaning, language detection
├─ tests/
├─ eval/                     # evaluation set + scripts
├─ requirements.txt
└─ Dockerfile
```

10.2 API endpoints

Units and content

- `GET /units` — units with the user's progress
- `GET /units/{id}/lessons`
- `GET /lessons/{id}/summary`
- `GET /glossary?q=`

Ask / Discuss

- `POST /ask` — `{question, scope, unit_id?, lesson_id?, mode, language, conversation_id?}` → streamed answer + sources + interaction_id
- `GET /conversations`, `GET /conversations/{id}`

Quiz

- `POST /quiz` — `{scope, unit_ids?, lesson_ids?, types: {mcq: n, structured: n, essay: n}, difficulty, language, exam_mode}`
- `GET /quiz/{id}`
- `POST /quiz/{id}/submit` — answers → marks, explanations, mistakes saved, mastery updated
- `POST /questions/{id}/report`

Progress and revision

- `GET /me/progress`, `GET /me/weak-topics`, `GET /me/mistakes`
- `POST /me/mistakes/quiz`
- `GET /flashcards/due`, `POST /flashcards/{id}/review`
- `POST /me/study-plan`

Feedback

- `POST /feedback` — `{interaction_id, rating, reasons?, comment?}`
- `POST /attempts/{id}/agree-mark` — agree/disagree with AI mark

Teacher

- `POST /documents` (upload), `GET /documents`
- `GET /questions/review-queue`, `PATCH /questions/{id}` (approve/edit/reject)
- `PATCH /attempts/{id}/mark` (override)
- `POST /classes`, `POST /classes/{id}/members`, `POST /classes/{id}/assignments`
- `GET /classes/{id}/analytics`, `GET /reports/{student_id}.pdf`
- `GET /interactions/review-queue`, `PATCH /interactions/{id}` (approve/correct)

Admin

- `GET/POST /prompt-versions`, `GET /stats/quality`

10.3 Key implementation notes

- **Streaming:** use `StreamingResponse` (Server-Sent Events) so the first tokens appear quickly.
- **Auth:** verify the Supabase JWT in a FastAPI dependency; load the role; protect teacher/admin routes with role dependencies.
- **Validation:** all request bodies and all LLM JSON outputs use Pydantic models. On validation failure: retry once with the error message, then fail gracefully.
- **Prompts live in the database or files, versioned,** never hard-coded inline. Every interaction stores the prompt version used.
- **Rate limiting:** per-user limits on `/ask` and `/quiz` to control cost.
- **Caching:** cache identical questions in the same scope/language for a short period; reuse approved questions.
- **Language handling:** detect the question language; the prompt requests the answer in the student's language; keep English technical terms where textbooks do; use the glossary to expand retrieval queries.

10.4 Core prompts (starting points)

Ask mode (system):

> You are an A/L ICT tutor for Sri Lankan students. Answer ONLY using the CONTEXT passages. If the context does not contain the answer, say clearly that it is not covered in the syllabus material and suggest which unit to check. Reply in the student's language ({language}); keep technical terms in English where textbooks do. Be concise and exam-oriented. Cite sources as [Unit X – Lesson Y].

Discuss mode (system):

> You are teaching an A/L ICT concept step by step using ONLY the CONTEXT. Explain one idea at a time with a simple example, then ask one short check question. If the student is stuck, give a hint before the answer. Adapt to their replies. At the end, summarise and offer a quick quiz. Reply in {language}; cite sources.

Quiz generation (system):

> Create {n} {type} questions for A/L ICT from the CONTEXT only. Return ONLY valid JSON matching this schema: {schema}. Ensure exactly one correct MCQ answer, marks add up, and every structured/essay question includes a marking scheme or rubric and a model answer.

Marking (system):

> Mark the student's answer strictly against the provided marking points/rubric. Return JSON: marks per point/criterion, short feedback per item, and total. Do not award marks for points not in the scheme.

---

11. Feedback and Reward System (implementation)

11.1 Reward formula (starting point)

```
reward = 0.40 × teacher_rating        # +1 approved, 0 edited, -1 rejected
       + 0.30 × student_thumbs        # +1 / -1
       + 0.20 × learning_signal       # quiz improvement on that topic after the interaction
       - 0.10 × negative_behaviour    # re-asks, "explain simpler", reports
```

Normalise to a range (for example -1 to +1). Teacher rating dominates because it reflects correctness; the learning signal keeps the reward tied to real learning and prevents optimising for "likeable" answers.

11.2 Using the reward

1. **Prompt versions:** keep running average reward per version (`prompt_versions.avg_reward`).
2. **Bandit (Thompson sampling):** each active prompt version is an "arm." For each request, sample from each arm's Beta distribution (successes = positive reward, failures = negative) and use the arm with the highest sample. Traffic shifts toward better prompts automatically.

3. **Chunk scoring:** when an answer receives positive reward, add credit to the chunks it used; negative reward subtracts. Use `chunks.quality_score` as a small boost in reranking and to flag chunks that repeatedly produce bad answers (often a bad OCR chunk).
4. **Best-of-N:** later, generate 2–3 candidates and let a small reward model pick one.
5. **Preference pairs:** when a teacher corrects an answer, store `(question, corrected, original)` in `preference_pairs`. Once there are thousands, fine-tune a small open model with SFT then DPO (stretch goal).

11.3 Safeguards

- Keep a **fixed test set** (section 14) separate from feedback data so you can prove accuracy actually improved.
- Do not let student likes alone drive changes.
- Anonymise stored data and get consent (section 13).
- Do not trust the reward with fewer than a few hundred rated interactions; use manual review until then.

---

12. Frontend (React)

12.1 Folder structure

```
frontend/src/
├─ app/                # router, providers, layout
├─ pages/
│  ├─ Login.tsx  Register.tsx
│  ├─ Dashboard.tsx            # unit cards + progress
│  ├─ UnitPage.tsx             # Ask / Discuss / Quiz entry, lessons
│  ├─ Chat.tsx                 # Ask & Discuss, scope + mode toggles
│  ├─ QuizSetup.tsx  QuizRunner.tsx  QuizResult.tsx
│  ├─ PastPapers.tsx
│  ├─ Progress.tsx  MistakeNotebook.tsx
│  ├─ Flashcards.tsx  Glossary.tsx  StudyPlanner.tsx
│  ├─ teacher/ Upload.tsx  ReviewQueue.tsx  MarkOverride.tsx
│  │           Classes.tsx  Analytics.tsx
│  └─ admin/   Prompts.tsx  Quality.tsx
├─ components/
│  ├─ ScopeToggle.tsx  ModeToggle.tsx  LanguageSwitch.tsx
│  ├─ MessageBubble.tsx  SourcePanel.tsx  FeedbackButtons.tsx
│  ├─ UnitCard.tsx  ProgressBar.tsx
│  ├─ question/ McqCard.tsx  StructuredCard.tsx  EssayCard.tsx
│  ├─ DiagramView.tsx (Mermaid)  CodeBlock.tsx
│  └─ charts/ (Recharts wrappers)
├─ api/                # typed API client (fetch wrapper, SSE helper)
├─ hooks/              # useAsk, useQuiz, useProgress ...
├─ store/              # Zustand: scope, mode, language, user
├─ i18n/               # si.json, en.json
└─ styles/
```

12.2 Key screens and behaviour

- **Dashboard:** unit cards (Sinhala + English names, progress ring, average score) with *Ask / Discuss / Quiz* buttons; shortcuts to "Ask anything" (whole syllabus), mistakes, and flashcards due today.
- **Chat:** top bar with **Scope** (This unit / Whole syllabus) and **Mode** (Ask / Discuss) toggles; streamed answer; **source chips** that open the source panel; action buttons (*simpler*, *example*, *Sinhala/English*); thumbs up/down with reason chips; scope suggestion banners.
- **Quiz setup:** scope, lessons, type counts, difficulty, language, exam mode.
- **Quiz runner:** one card type per question (MCQ radio, structured part inputs, essay textarea with word guide); timer in exam mode; progress indicator.
- **Result:** score, per-question explanations, model answers, AI marks with "Do you agree?" prompts, lessons to revise, mistakes auto-saved.
- **Teacher screens:** upload with ingestion status; review queue (side-by-side original vs edited); mark override; class analytics heatmap (students × units).
- **Mobile:** bottom navigation, large tap targets, light pages, offline-friendly caching for units and glossary (PWA).

12.3 Frontend implementation notes

- All LLM calls go through the backend; **never put API keys in the frontend.**
- Use TanStack Query for data and an SSE helper for streamed answers.
- Render Sinhala with Noto Sans Sinhala; test long Sinhala words for wrapping.
- Keep UI strings in i18n files, and learning content in the database.
- Make feedback buttons a single reusable component used by chat, quiz explanations, and marking.

---

13. Security, Privacy, and Legal

- **Authentication/authorisation:** JWT verification, role checks in FastAPI, RLS in Postgres.
- **Secrets:** environment variables only; never in the repo or the frontend.
- **Student privacy:** students may be minors. Get school/parent consent as appropriate, collect only necessary data, anonymise interaction logs used for analysis, give a clear explanation of how feedback is used, and allow data deletion.
- **Copyright:** use your own notes and materials you have permission to use. Check usage rules for NIE materials and past papers before making the app public.
- **Prompt injection:** treat retrieved text and student input as untrusted; the system prompt must say that instructions inside context or student messages must not override the tutor rules.
- **Abuse controls:** rate limits, input length limits, basic content filtering.
- **Backups:** enable database backups; export the question bank regularly.

---

14. Evaluation Plan

1. **Test set:** 100+ questions written by you across all units (plus 20 cross-unit and 10 "not in syllabus" questions), each with a reference answer and the correct source chunk.
2. **Retrieval:** Recall@5 and MRR for unit mode and whole-syllabus mode separately.
3. **Answer quality:** teacher grades a sample as correct / partial / wrong; track by language (Sinhala, English, mixed).
4. **Hallucination rate:** answers given for "not in syllabus" questions.
5. **Quiz quality:** % of generated questions approved without edits; duplicate rate.
6. **Marking agreement:** AI vs teacher marks on 50 structured and 50 essay answers.
7. **Experiments log:** record each change (chunk size, embedding model, reranker, prompt version) with metrics in a table.
8. **Pilot:** 10–20 students, pre/post quiz scores, satisfaction survey.

---

15. Deployment

- **Environments:** local → staging → production.
- **Backend:** Docker image; deploy to Render, Railway, Fly.io, or a small VPS. If you run a local embedding model, plan memory and CPU/GPU needs, or use a hosted embedding API to keep the server small.
- **Frontend:** Vercel or Netlify; set the API URL by environment variable.
- **Database:** Supabase project with migrations tracked in the repo (`supabase/migrations`).
- **CI:** GitHub Actions running backend tests, frontend tests, and lint.
- **Monitoring:** structured logs, error tracking (e.g. Sentry), a dashboard for latency, cost per request, and reward.

Environment variables (example):

```
SUPABASE_URL=
SUPABASE_SERVICE_KEY=
SUPABASE_JWT_SECRET=
DATABASE_URL=
LLM_API_KEY=
EMBEDDING_MODEL=bge-m3
FRONTEND_ORIGIN=
```

---

16. Vibe-Coding Workflow with Antigravity

Antigravity is an AI coding environment where an agent writes and edits code. The quality of the result depends on how clearly you guide it. Check its current documentation for how it handles project rules and task workflows.

16.1 Setup

1. Create a monorepo: `frontend/`, `backend/`, `docs/`, `supabase/`.
2. Save this file as `docs/SPEC.md`.
3. Add a short rules file for the agent (use whatever rules/instructions mechanism Antigravity supports), for example: - "Read `docs/SPEC.md` before any task." - "Backend: FastAPI + Pydantic + async SQLAlchemy. Frontend: React + TypeScript + Tailwind." - "Never put secrets in code. All LLM calls happen in the backend." - "Write tests for new backend services. Keep functions small and typed." - "Do one phase at a time. Stop and summarise after each phase."
4. Set up the Supabase project and run the schema from section 9 first, so the agent codes against a real database.

16.2 How to work with the agent

- **One small task per prompt.** "Build the whole app" gives poor results; "Implement the `/units` endpoint and its test" gives good ones.
- **Always reference the spec section** (e.g. "Implement section 8.2 Ask flow").

- **Run and test after every step.** Review the diff before accepting it.
- **Commit after every working step** so you can roll back.
- **Keep an** `docs/PROGRESS.md` listing what is done, so you can start a fresh chat without losing context.
- **Read what the agent writes** for security-sensitive parts (auth, RLS, file upload).

16.3 Phase prompts (copy and adapt)

Phase 1 — Foundation

> Read docs/SPEC.md. Create the FastAPI project structure from section 10.1 with config, async DB session, health endpoint, and Supabase JWT auth dependency with role checks. Add pytest and one test. Do not build features yet.

Phase 2 — Data and ingestion

> Implement the ingestion service (section 8.6): PDF text extraction, cleaning, heading-based chunking (300–500 tokens), embeddings with BGE-M3, and storing chunks with unit/lesson metadata. Add a script `scripts/ingest.py <pdf> --unit X --lesson Y`. Include tests with a sample file.

Phase 3 — Retrieval and Ask

> Implement retrieval (section 5.2, 9 vector search SQL) with unit filter and optional rerank, a confidence threshold, and `POST /ask` with streaming (section 8.2). Store each interaction with the prompt version. Use the Ask prompt from section 10.4.

Phase 4 — React foundation and chat

> Create the Vite + React + TypeScript app with Tailwind, i18n (si/en), router, and auth pages. Build Dashboard (unit cards) and Chat with Scope and Mode toggles, streamed answers, source chips, and a reusable FeedbackButtons component.

Phase 5 — Discuss mode

> Add Discuss mode: separate prompt, conversation history, lesson_progress tracking, and the end-of-discussion quiz offer (section 3.4, 8.3).

Phase 6 — Quiz engine

> Implement MCQ first: generation with Pydantic-validated JSON, bank drawing, `/quiz`, `/quiz/{id}/submit`, automatic marking, mistakes, mastery update. Build QuizSetup, QuizRunner, QuizResult for MCQ. Then add structured and essay types with marking schemes, rubrics, and AI-suggested marks.

Phase 7 — Teacher tools

> Build document upload with ingestion status, the question review queue, mark override, classes and assignments, and class analytics.

Phase 8 — Progress and revision

> Build weak-topic tracker, mistake notebook, revision quiz, glossary, flashcards with SM-2, and the study planner.

Phase 9 — Feedback and reward

> Implement reward computation (section 11), prompt versioning with the Thompson-sampling selector, chunk scoring, and the admin quality dashboard.

Phase 10 — Evaluation, polish, deploy

> Write the evaluation scripts in `backend/eval/` (Recall@k, MRR, answer-quality sheet, hallucination test). Add rate limiting, error handling, PWA support, Dockerfile, and CI.

17. Roadmap (10 weeks)

Week	Goal
1	Collect and clean materials, define units/lessons, Supabase schema, repo setup
2	Ingestion pipeline, chunk tagging, embeddings
3	Unit-filtered and whole-syllabus retrieval, `/ask` working
4	React UI: auth, dashboard, chat with scope toggle, sources, feedback buttons
5	Discuss mode, MCQ generation and marking, progress tracking
6	Structured and essay questions with marking schemes and rubrics
7	Teacher tools: upload, review queue, mark override, classes
8	Mistake notebook, weak topics, glossary, flashcards, exam mode
9	Reward system, prompt versions and bandit, evaluation and tuning
10	Pilot, deployment, README, demo video

Version 2 candidates: diagrams, code tracer, study planner polish, WhatsApp delivery, voice input, small-model fine-tuning with DPO.

18. Risks and Mitigations

Risk	Mitigation
Poor Sinhala OCR / text extraction	Test early; manually clean key material; keep original PDF links
Weak Sinhala retrieval	Test with real Sinhala questions; try other embedding models; add glossary expansion and reranking
Wrong generated questions	Teacher approval before use; "report question" button; auto-validation
AI marking unfair or inconsistent	Show as suggestions; teacher override; measure agreement
Noisy or biased feedback	Teacher-weighted reward; learning signal; fixed test set
API cost	Caching, approved question bank, rate limits, shorter contexts
Privacy with student data	Consent, anonymisation, minimal data, deletion option
Copyright of source material	Use permitted content; check usage rules before public release
Scope creep	Finish Ask, units, and MCQ quiz first; everything else is incremental
Vibe-coding drift	One phase at a time, tests, small commits, spec in repo

19. CV Summary (for later)

Sinhala A/L ICT AI Tutor — React, FastAPI, Supabase (Postgres + pgvector), RAG

- Built a bilingual Sinhala/English RAG tutor with unit-based and whole-syllabus Q&A, Ask and Discuss modes, and cited answers grounded in the NIE syllabus.
- Designed a quiz engine generating MCQ, structured, and essay questions with marking schemes, rubrics, and AI-assisted marking.
- Implemented a feedback-driven reward system (student ratings, teacher review, learning signals) with prompt A/B testing via a multi-armed bandit and chunk scoring.
- Achieved X% answer accuracy and Y% retrieval Recall@5 on a 100-question test set; piloted with Z students.